Industrial Internship Report on

"Predictive Maintenance of Gearbox Using Vibration Sensors"

Prepared by

Sneha Dhanawade

Executive Summary

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

This internship was focused on a project/problem statement provided by UCT. The project, including this report, was completed over 4 weeks.

My project was "Predictive Maintenance of Gearbox Using Vibration Sensors" - building an end-to-end machine learning pipeline that classifies a gearbox as Healthy or having a Broken Tooth from vibration sensor signals, achieving 99.1% accuracy on held-out test data, and packaging it behind an interactive dashboard with database-backed prediction logging.

This internship gave me a very good opportunity to get exposure to industrial problems and design/implement a solution for them. It was an overall great experience to have this internship.

TABLE OF CONTENTS

Right-click the table above and choose "Update Field" in Microsoft Word to populate page numbers.

1 Preface

This report summarizes 4 weeks of work carried out as part of the Industrial Internship Program conducted by upskill Campus (USC) and The IoT Academy, in collaboration with UniConverge Technologies Pvt Ltd (UCT).

Practical, industry-relevant internships are an important part of career development, as they bridge the gap between academic learning and real-world engineering problems. They provide exposure to how a problem is scoped, how data is actually messy and needs cleaning, and how a solution is iterated on and evaluated honestly rather than just theoretically.

My project, "Predictive Maintenance of Gearbox Using Vibration Sensors", deals with detecting gearbox faults - specifically a broken gear tooth - from vibration sensor data before they cause complete equipment failure. This is a core concern for manufacturers who want to lower maintenance costs, extend equipment life, reduce downtime, and improve production quality by addressing problems before they cause equipment failures.

This opportunity was given by USC and UCT as part of their structured internship program.

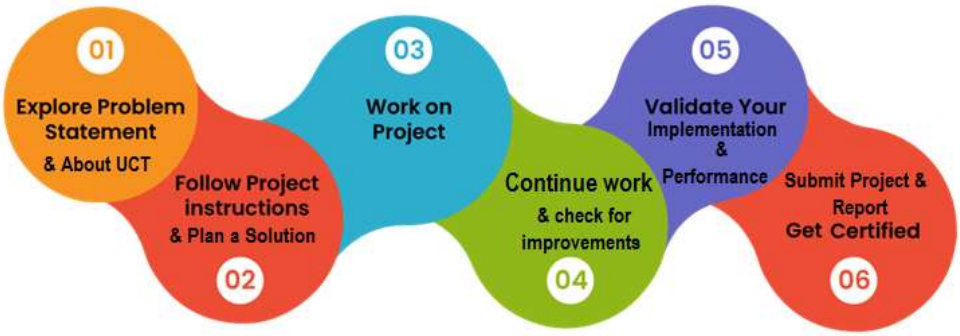

Figure 1: Internship program structure (USC / UCT)

The 4-week program was planned and executed as follows:

- Week 1: Dataset exploration (SpectraQuest Gearbox Fault Diagnostics dataset) and Python environment setup.
- Week 2: Statistical feature extraction from raw vibration signals, building the feature datasets (whole-file and windowed approaches).
- Week 3: Machine learning fundamentals, model training (Random Forest), and saving the trained model for reuse.
- Week 4: UI design and integration - building an interactive Streamlit dashboard connected to the trained model and a SQLite database for prediction logging.

Through this internship I learned how to move from raw sensor data to a deployable machine learning solution end-to-end, including a very practical lesson in avoiding data leakage and validating a model honestly rather than trusting a suspiciously perfect accuracy score obtained from too little data.

I would like to thank the mentors at UCT and upskill Campus for their guidance and support throughout this internship, and for structuring the program in a way that built up skills progressively week by week.

To my juniors and peers taking up this internship: don't be discouraged when your first model shows a suspiciously perfect result - debugging exactly why is where the real learning happens in this field.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

UniConverge Technologies Pvt Ltd (UCT) is a company established in 2013, working in the Digital Transformation domain and providing industrial solutions with a prime focus on sustainability and Return on Investment (RoI).

For developing its products and solutions, UCT leverages various cutting-edge technologies, e.g., Internet of Things (IoT), Cyber Security, Cloud Computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, and Front-end development.

i. UCT IoT Platform (UCT Insight)

UCT Insight is an IoT platform designed for quick deployment of IoT applications while providing valuable "insight" for a business process. It is built in Java for the backend and ReactJS for the front end, with support for MySQL and various NoSQL databases. It enables device connectivity via industry-standard IoT protocols (MQTT, CoAP, HTTP, Modbus TCP, OPC UA) and supports both cloud and on-premises deployments, with features for custom dashboards, analytics and reporting, alerts and notifications, third-party integrations (Power BI, SAP, ERP), and a rule engine.

ii. Smart Factory Platform (Factory Watch)

Factory Watch is a platform for smart factory needs. It gives users a scalable solution for production and asset monitoring, OEE and predictive maintenance solutions that scale up to a digital twin for their assets, and helps identify and improve KPIs from machine-generated data via a modular, SaaS-based architecture.

iii. LoRaWAN-based Solutions

UCT is one of the early adopters of LoRaWAN technology, providing solutions in Agritech, Smart Cities, Industrial Monitoring, Smart Street Lighting, and Smart Water/Gas/Electricity metering.

iv. Predictive Maintenance

UCT provides industrial machine health monitoring and predictive maintenance solutions leveraging embedded systems, Industrial IoT, and Machine Learning technologies to find the Remaining Useful Life (RUL) of various machines used in production processes - the broader technology area this internship project falls under.

2.2 About upskill Campus (USC)

upskill Campus, along with The IoT Academy and in association with UniConverge Technologies, has facilitated the smooth execution of the complete internship process. USC is a career development platform that delivers personalized executive coaching in an affordable, scalable, and measurable way.

2.3 The IoT Academy

The IoT Academy is the EdTech division of UCT, running long executive certification programs in collaboration with EICT Academy, IITK, IITR, and IITG across multiple domains.

2.4 Objectives of this Internship Program

The objectives of this internship program were to:

- Get practical experience of working in the industry.
- Solve real-world problems.
- Have improved job prospects.

- Have an improved understanding of my field and its applications.
- Have personal growth in areas like communication and problem-solving.

2.5 Reference

- [1] SpectraQuest Gearbox Fault Diagnostics Simulator - dataset documentation.
- [2] Keller, J.; Wallen, R. "Gearbox Reliability Collaborative Phase 3 Gearbox 2 Test Report", NREL/TP-5000-63693, National Renewable Energy Laboratory, 2015. (Used as background reading on gearbox vibration/instrumentation behaviour under load.)
- [3] scikit-learn documentation - RandomForestClassifier, GroupShuffleSplit.
- [4] Streamlit documentation.

2.6 Glossary

Term	Meaning
RMS	Root Mean Square - a measure of the overall energy/magnitude of a vibration signal.
Kurtosis	Statistical measure of "peakiness" of a signal distribution; rises sharply for impulsive fault signals.
Crest Factor	Ratio of peak value to RMS value; a high value indicates impulsive, spiky signal behaviour.
Windowing	Splitting a long signal into smaller fixed-length segments before feature extraction.
Data Leakage	When information from the test set inadvertently influences model training, giving an unrealistically high accuracy.
Random Forest	An ensemble machine learning algorithm that combines many decision trees for classification.
UI	User Interface - here, the Streamlit-based interactive dashboard built for this project.

3 Problem Statement

Gearboxes in industrial and rotating machinery are prone to failures that are costly to repair and cause significant unplanned downtime. Traditional maintenance approaches are either reactive (repairing only after a failure occurs - expensive and disruptive) or purely schedule-based preventive maintenance (replacing parts at fixed intervals regardless of actual condition - often wasteful).

The assigned problem statement (Project 8) was:

- Use vibration signals recorded by 4 sensors, placed in four different directions, from SpectraQuest's Gearbox Fault Diagnostics Simulator.
- Cover a range of operating loads, from 0% to 90%.
- Distinguish between two gearbox conditions: Healthy and Broken Tooth.
- Build a system that can automatically flag a faulty gearbox from its vibration signature, enabling maintenance to be scheduled before a complete failure occurs - i.e., predictive rather than reactive or purely schedule-based maintenance.

This directly aligns with UCT's Predictive Maintenance solution area described in Section 2.1, which uses vibration monitoring and Machine Learning to estimate machine health and remaining useful life.

4 Existing and Proposed Solution

Existing Solutions

- Manual/scheduled inspection: technicians periodically inspect gearboxes, which is labour-intensive and cannot catch faults that develop between inspection intervals.
- Threshold-based vibration monitoring: some systems flag a fault when raw vibration amplitude crosses a fixed threshold. This is prone to false alarms because vibration amplitude also increases naturally with load, and it does not use the richer statistical/frequency information present in the signal shape.
- Published instrumentation studies (e.g., NREL's Gearbox Reliability Collaborative test reports) document how gearbox vibration and bearing responses behave under different loading conditions on real drivetrains, providing useful domain context, but such work is typically focused on detailed mechanical instrumentation and analysis rather than an automated ML classifier.

Proposed Solution

- Extract a compact set of statistical features - mean, standard deviation, RMS, kurtosis, skewness, crest factor, and peak-to-peak - from each of the 4 sensor signals. These are established indicators of impulsive fault behaviour; for example, kurtosis and crest factor rise sharply when a broken tooth causes a sharp impact once per rotation.
- Segment each raw recording into smaller time windows (2048 samples each) rather than treating a whole ~88,000-sample file as a single data point. This increases the usable sample count from 20 (whole-file) to 978 (windowed) and supports a statistically meaningful, group-aware train/test evaluation.
- Train a Random Forest classifier on these features to automatically classify a gearbox recording as Healthy or Broken Tooth.
- Package the trained model behind an interactive Streamlit dashboard so that a new vibration recording can be uploaded and classified instantly, with every prediction logged to a SQLite database for traceability.

Value Addition

- Moves from manual/threshold-based inspection to an automated, data-driven classifier.
- Captures early-stage impulsive fault signatures (via kurtosis/crest factor) that a simple amplitude threshold would miss.
- Provides a working, demonstrable interface rather than only an offline analysis notebook - closer to how such a system would actually be used for monitoring on a factory floor.
- Explicitly guards against a classic pitfall (data leakage / too little data) by using windowed features with a group-aware split, rather than reporting an inflated accuracy number.

4.1 Code Submission (GitHub link)

https://github.com/snehaxhub/upskillcampus/blob/main/upskillcampus_repo/GearboxPredictiveMaintenance.py

4.2 Report Submission (GitHub link)

https://github.com/snehaxhub/upskillcampus/blob/main/GearboxPredictiveMaintenance_Sneha_USC_UCT.pdf

5 Proposed Design / Model

The solution is organized as a pipeline: raw sensor data flows through feature extraction, into model training, and the resulting saved model is served through an interactive UI, with every prediction logged to a database.

5.1 High Level Diagram

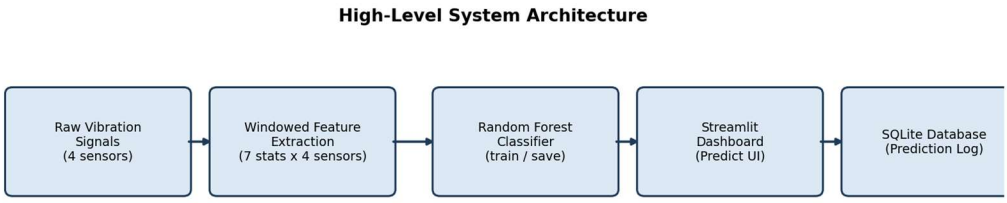

Figure 2: High-level system architecture

5.2 Low Level Diagram

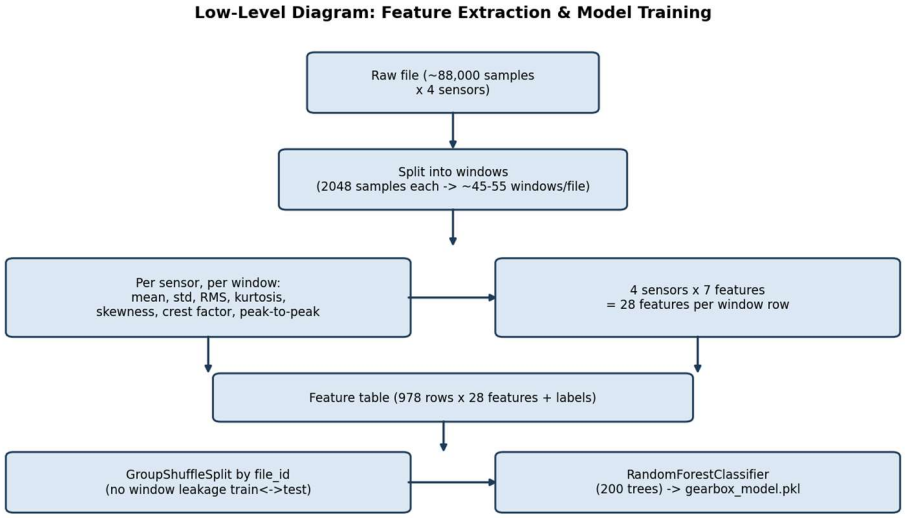

Figure 3: Feature extraction and model training detail

Design notes:

- feature_utils.py is shared by both the training script and the UI, so predictions are always computed the same way the model was trained - this was a deliberate design decision to avoid train/serve skew.
- Windows from the same original file are kept together within either the training or the test split (never split across both), using a file_id group key with scikit-learn's GroupShuffleSplit. This avoids the model "cheating" by seeing near-duplicate windows from the same recording in both sets.
- The final deployed model is re-trained on all 978 samples after validating performance on the held-out split, which is standard practice once test performance has been verified.

5.3 Interfaces

A Streamlit-based dashboard (industrial dark theme) was built as the user interface, with four pages:

- Overview - project summary, model accuracy, and confusion matrix.

- Predict - upload a raw vibration file (or use a demo sample) to get an instant Healthy/Broken Tooth prediction with confidence, a signal plot, and key feature values. Every prediction is logged to the database.
- Explore Dataset - interactive plots comparing feature distributions between healthy and broken-tooth samples, across load levels.
- Prediction History - table and pie chart of every prediction logged to the database so far.

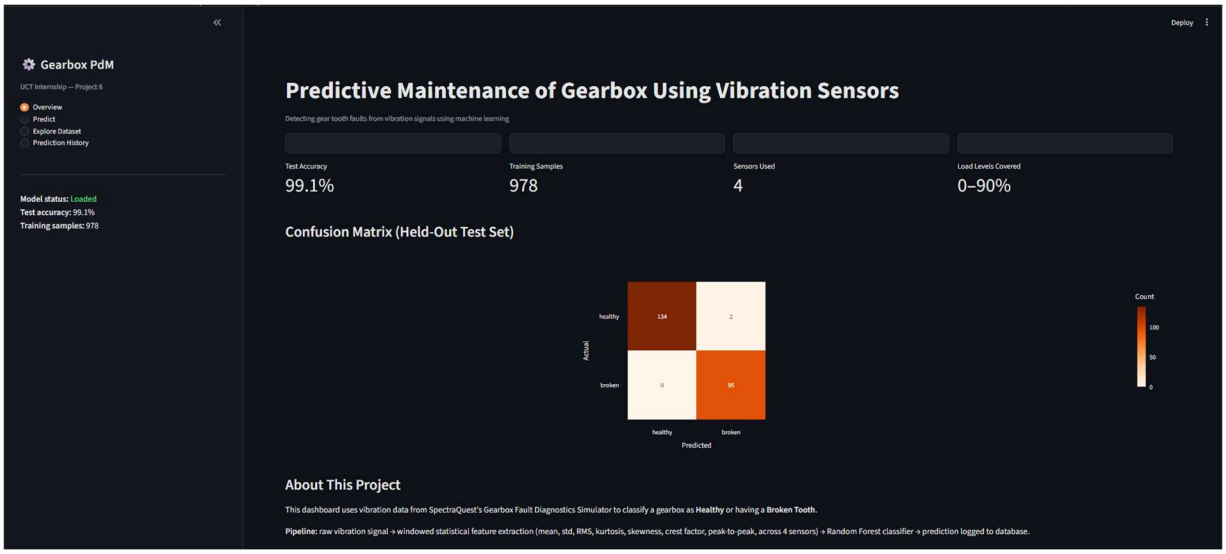

Figure 4: Dashboard - Overview page

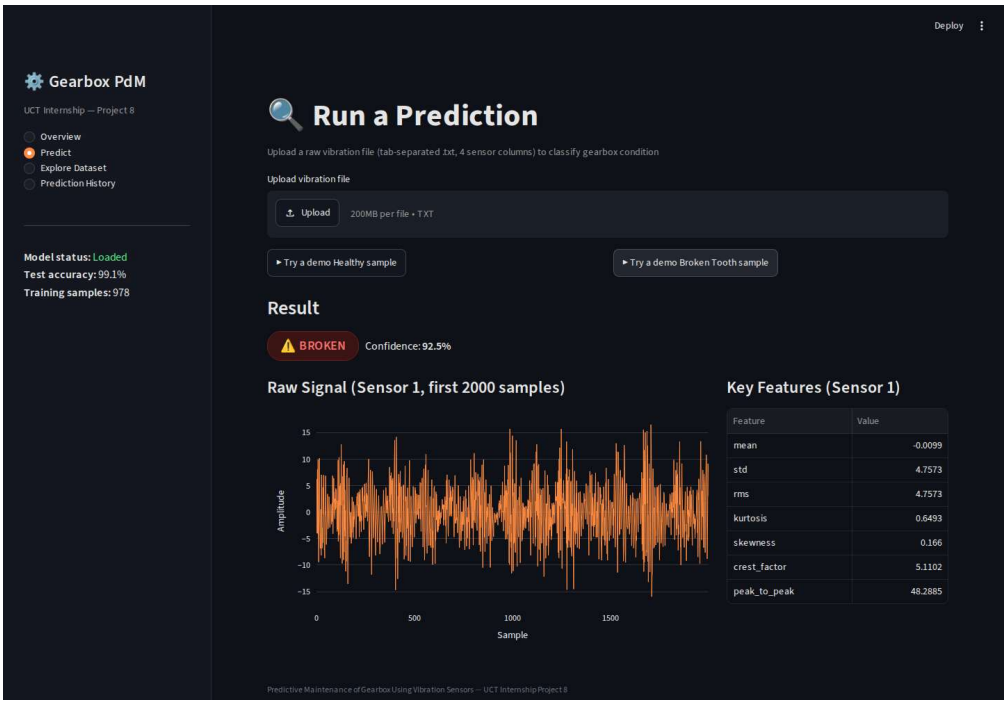

Figure 5: Dashboard - Predict page, correctly classifying a demo Broken Tooth sample

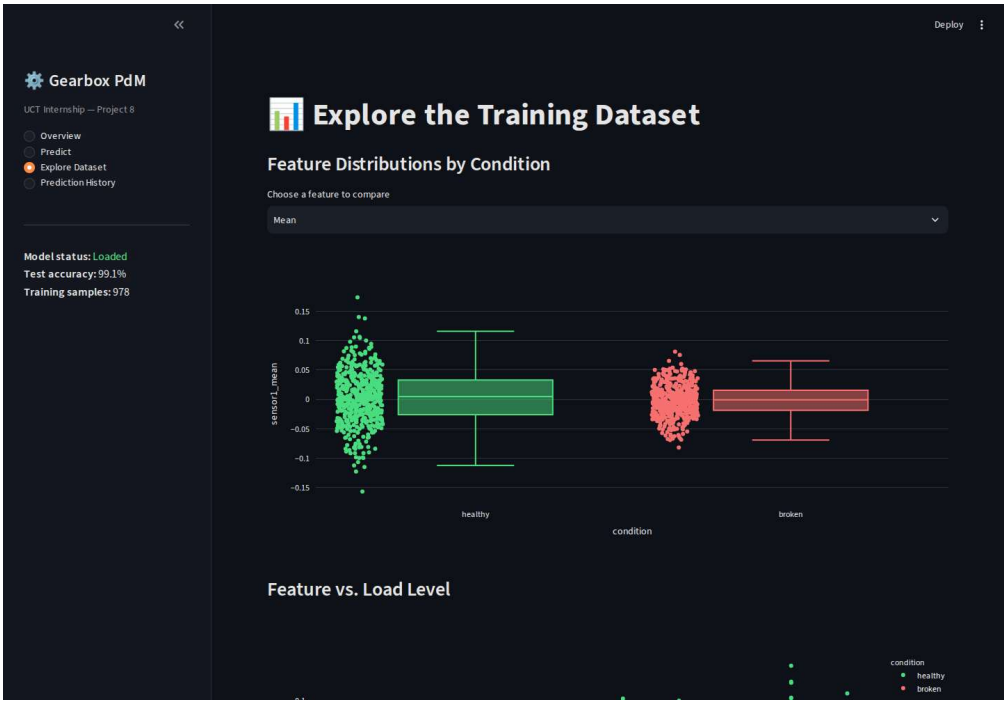

Figure 6: Dashboard - Explore Dataset page

6 Performance Test

This section evaluates whether the solution genuinely works, rather than only looking correct on paper.

6.1 Test Plan / Test Cases

Key constraints identified for this project:

- Data volume: only 20 raw recordings are available in total (10 Healthy + 10 Broken Tooth, one per load level 0-90%). Evaluating a model on 20 whole-file samples is not statistically trustworthy.
- Class balance: after windowing, the dataset is well balanced (492 healthy windows vs. 486 broken-tooth windows).
- Data leakage risk: naive windowing followed by a random row-wise train/test split would let near-identical windows from the same file appear in both sets, inflating accuracy artificially.
- Inference speed: for the dashboard to be usable interactively, feature extraction and prediction for one uploaded file must complete in well under a few seconds.

Test cases were designed to check: (a) whether windowing genuinely increases usable sample count, (b) whether accuracy holds up under a group-aware (leakage-free) split, and (c) whether the saved/reloaded model gives the same predictions as the freshly trained one.

6.2 Test Procedure

- Baseline (whole-file) test: 20 samples, Leave-One-Out Cross-Validation, RandomForestClassifier(n_estimators=100).
- Improved (windowed) test: each raw file split into 2048-sample windows -> 978 samples total, split 75/25 by file_id using GroupShuffleSplit (747 train / 231 test), RandomForestClassifier(n_estimators=200) trained on the training split only, then evaluated on the held-out test windows (from files never seen during training).
- The final model was re-trained on all 978 samples, saved via joblib, and reloaded in the Streamlit app to confirm it reproduces the same predictions.

6.3 Performance Outcome

The whole-file baseline (20 samples) scored a perfect 100% accuracy under Leave-One-Out Cross-Validation. This was treated as a red flag rather than a success, since a perfect score from only 20 samples is a classic sign of too little data to trust, not genuine generalization.

The windowed approach gave a far more defensible result:

- Test accuracy: 99.13% (229 / 231 correct) on files never seen during training.
- Precision/Recall: 1.00 / 0.99 for Healthy, 0.98 / 1.00 for Broken Tooth.
- Only 2 misclassifications out of 231 test windows, both healthy windows predicted as broken (no broken windows were missed).

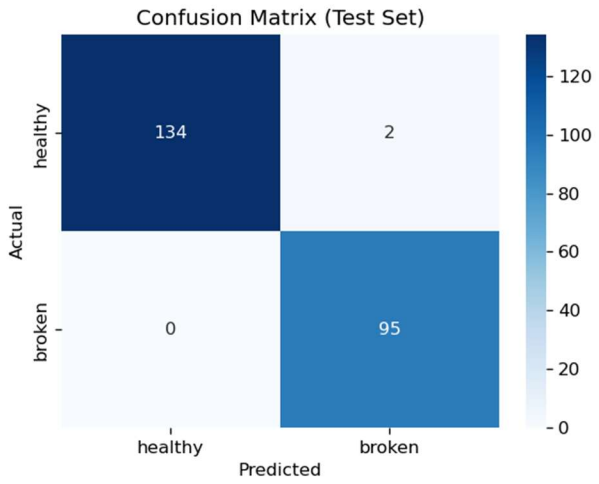

Figure 7: Confusion matrix on the held-out test set

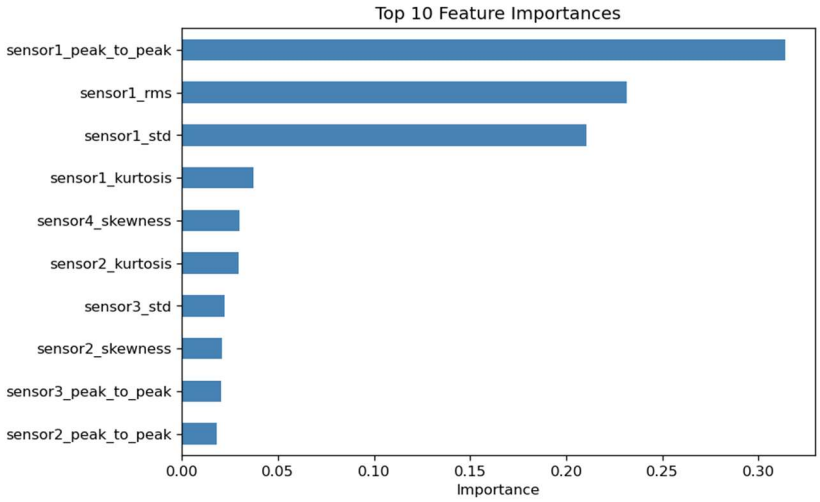

Figure 8: Top 10 most important features driving the model's predictions

Sensor 1's peak-to-peak, RMS, and standard deviation were the most influential features - consistent with the time-domain and feature-separation plots below, which show visibly different signal shapes and kurtosis/crest-factor values between the two conditions.

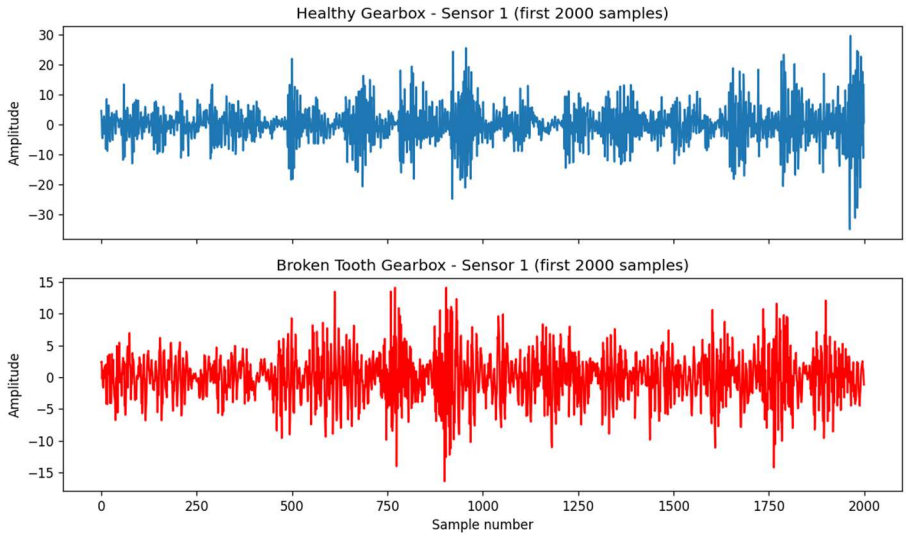

Figure 9: Raw time-domain signal - Healthy vs. Broken Tooth (Sensor 1)

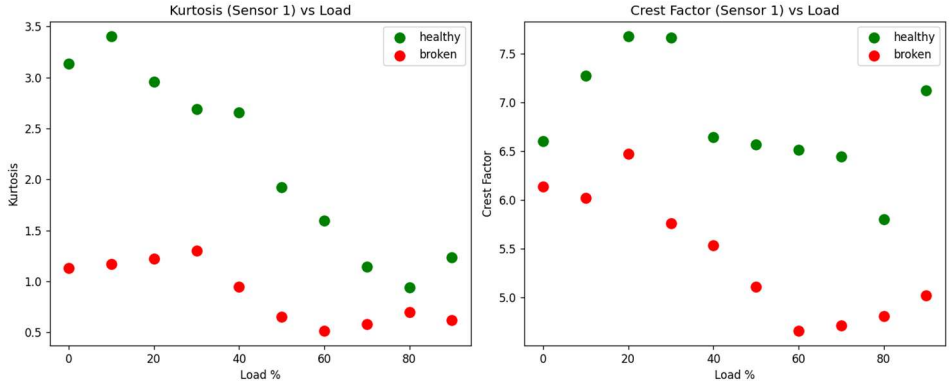

Figure 10: Kurtosis and crest factor separating Healthy vs. Broken Tooth across load levels

Constraint handling summary: the data-volume constraint was addressed via windowing; the leakage-risk constraint was addressed via group-aware splitting by file_id; inference speed was found to be well under one second per uploaded file in the dashboard, which is suitable for interactive use. A constraint not yet tested is generalization to shaft speeds or load conditions outside this dataset (only 30 Hz shaft speed, 0-90% load was available) - this is called out as future work in Section 8.

7 My Learnings

Over the 4 weeks of this internship, I moved from not knowing the dataset at all to having a working, demoable predictive maintenance system, and picked up several concrete lessons along the way:

- Understanding vibration data: how raw sensor signals are structured, and how to visually and statistically tell a healthy signal apart from a faulty one.
- Feature engineering for signals: learned why simple statistics like kurtosis and crest factor are effective fault indicators, rather than just feeding raw signals into a model.
- The importance of enough data and a fair evaluation: my very first model scored a perfect 100% accuracy on only 20 samples, which I initially thought was a good result. Investigating why it was suspicious - and fixing it via windowing plus a group-aware train/test split - was the single most valuable lesson of this internship, because it is a mistake that is easy to make and easy to miss.
- Machine learning fundamentals: how a Random Forest classifier is trained, evaluated, and saved/reloaded for reuse.
- Building a usable interface: connecting a trained model to a Streamlit dashboard, and designing pages that make the model's behaviour easy to inspect (Overview, Predict, Explore Dataset, Prediction History).
- Working with a database: using SQLite to log predictions, and understanding how this design would extend to a production database if needed.
- Debugging discipline: a large share of the effort in Weeks 2-4 was writing code, hitting errors, and carefully isolating and fixing them - a practical skill that does not come across in a finished project but was a significant part of the actual work.

These learnings extend well beyond this specific project: the workflow of exploring data, engineering features, honestly validating a model, and packaging it behind a usable interface is the same workflow used in real industrial machine learning projects, and this internship gave me hands-on practice with all of it.

8 Future Work Scope

Due to the time constraints of this internship, the following ideas were identified but not implemented, and are recommended as next steps:

- Frequency-domain features: apply FFT to the vibration signals and extract features around the gear-mesh frequency and its harmonics, which are expected to further improve fault sensitivity beyond the current time-domain statistical features.
- Compare additional models: try SVM, Gradient Boosting, or XGBoost against the current Random Forest baseline, and compare performance and inference speed.
- Broader operating conditions: this dataset only covers a single shaft speed (30 Hz); testing generalization across multiple speeds, and to fault types beyond a broken tooth (e.g., worn gear, bearing faults), would make the solution more production-ready.
- Production database: migrate the SQLite prediction log to PostgreSQL/MySQL to support a real multi-user deployment, as outlined in the project's README.
- Cloud deployment: deploy the Streamlit dashboard (e.g., via Streamlit Community Cloud) to obtain a public, shareable link for live demonstrations.
- Real-time streaming: extend the pipeline to accept a live/streaming feed from physical vibration sensors, rather than only uploaded files, to move closer to an actual industrial monitoring deployment.